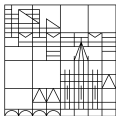


Global Illumination Methods

Practical Course

Bin Chen, Yumeng Xue

Universität
Konstanz



Starter Framework

Purpose

This starter project is a minimal but runnable ray-based renderer. The goal of this exercise is not to write a large amount of code, but to understand the rendering pipeline by completing a few missing steps.

Starter Framework

Main modules

- ▶ `io`: image output, model loading
- ▶ `app`: GUI, viewer, scene presets
- ▶ `core`: math-adjacent rendering basics
- ▶ `scene`: camera, scene container, hit record
- ▶ `geometry`: primitives and geometric hierarchies
- ▶ `shading`: materials, textures, lights
- ▶ `render`: renderer and integrators
- ▶ `accel`: acceleration structures

Rendering Pipeline

Scene

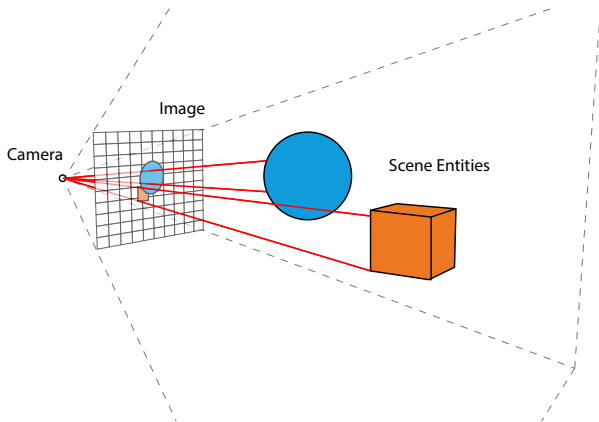
- Geometry & Materials
- Lights
- Camera

Renderer

- for each pixel / sample
- Camera generates ray
 - Scene finds closest hit
 - Integrator computes radiance
 - Direct light?
 - Shadow test?
 - Reflection / refraction?
 - Indirect bounce?
 - Volume effects?

Image

- accumulate
- gamma / tone map
- display / save



Minimal Rendering Pipeline

From pixel to shaded image

1. Renderer loops over image pixels
2. `camera.cpp` maps a pixel sample to a primary ray
3. `scene.cpp` tests the ray against all primitives
4. `sphere.cpp` computes sphere intersections and fills a `HitRecord`
5. `direct_lighting_integrator.cpp` evaluates direct light and shadow visibility
6. Viewer displays the final image in the GUI

Exercise task

Complete the TODOs in the four files above to reconstruct the full path:

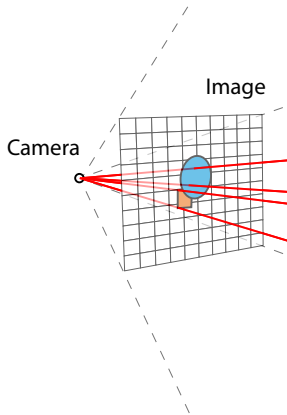
pixel -> ray -> hit -> shading -> image.

Fill-in Task

[src/scene/camera.cpp](#)

Convert image coordinates into screen coordinates and generate the primary ray.

```
1 Ray Camera::generateRay(double sampleX, double sampleY) const {  
2     // TODO: Convert sample coordinates (sampleX, sampleY)  
3     // to normalized screen coordinates (u, v) in the range [-1, 1].  
4     const double u = 0.0;  
5     const double v = 0.0;  
6  
7     // Ray direction  
8     glm::dvec3 dir =  
9         m_forward  
10        + u * m_halfWidth * m_right  
11        + v * m_halfHeight * m_up;  
12  
13    // Return a ray originating from the camera position and pointing in the  
14    // computed direction.  
15    return Ray(m_eye, glm::normalize(dir));  
16 }
```

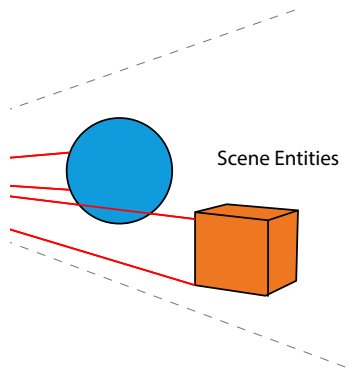


Fill-in Task

[src/scene/scene.cpp](#)

Traverse primitives and find the closest visible hit.

```
1 bool Scene::intersect(const Ray& ray, HitRecord& rec)
   const {
2     bool hitAnything = false;
3     double closestSoFar = ray.tMax;
4
5     // TODO: Implement ray-scene intersection
6     // by iterating over all primitives and finding the
       closest hit.
7
8     return hitAnything;
9 }
```

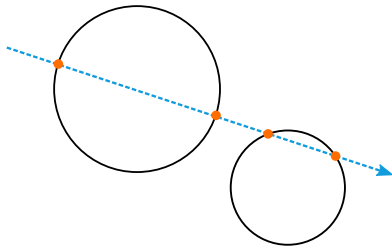


Fill-in Task

[src/geometry/sphere.cpp](#)

Implement ray-sphere intersection and write the hit data into HitRecord.

```
1 bool Sphere::intersect(const Ray& ray, HitRecord& rec
   ) const {
2     glm::dvec3 oc = ray.origin - m_center;
3
4     // TODO: Implement ray-sphere intersection and
       populate the HitRecord.
5
6     return false;
7 }
```



Fill-in Task

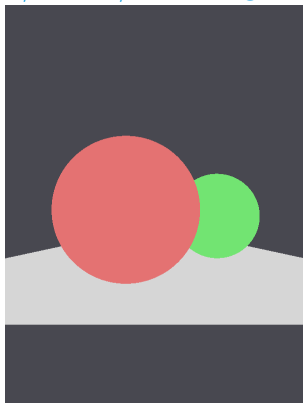
src/render/direct_lighting_integrator.cpp

Compute direct illumination from the point lights.

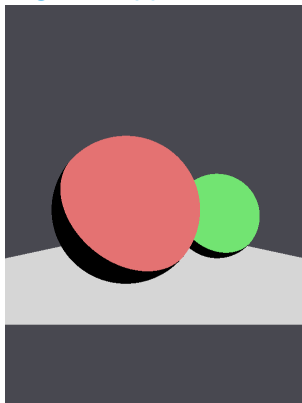
```
1   Color result = rec.material ? rec.material->emission(rec) : Color(0.0, 0.0, 0.0);
2
3   // wo is the outgoing direction toward the camera/viewer.
4   glm::dvec3 wo = -ray.direction;
5
6   // Traverse all lights in the scene.
7   for (const auto& lightBase : scene.lights()) {
8       // The starter scene only uses point lights.
9       auto pointLight = std::dynamic_pointer_cast<PointLight>(lightBase);
10      if (!pointLight) {
11          continue;
12      }
13
14      // TODO:
15      // Compute the contribution of this light source:
16      // determine the light direction, test visibility with a shadow ray,
17      // evaluate the material response, and accumulate the result.
18  }
```

Fill-in Task

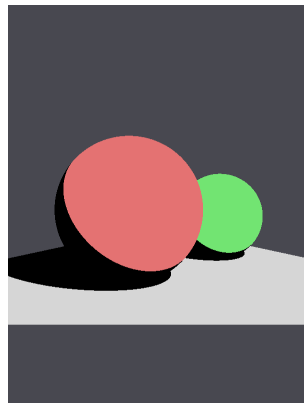
[src/render/direct_lighting_integrator.cpp](#)



Albedo



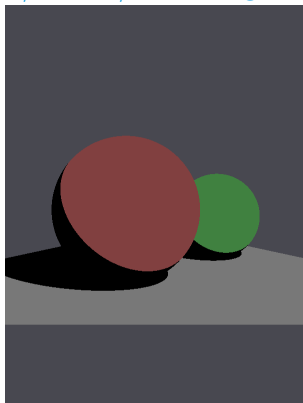
$n \cdot l$



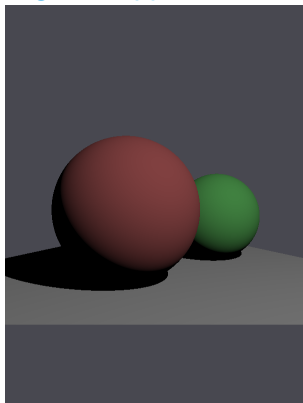
Shadow Visibility

Fill-in Task

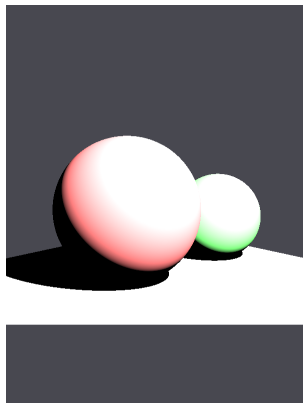
[src/render/direct_lighting_integrator.cpp](#)



BRDF



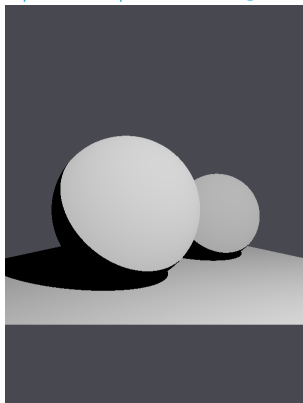
Cosine Term



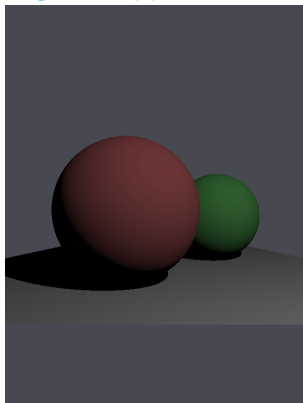
Light Intensity

Fill-in Task

[src/render/direct_lighting_integrator.cpp](#)



Distance Attenuation



Final Direct Lighting Result

Fill-in Task

Complete the following files

- ▶ `src/scene/camera.cpp`
Convert image coordinates into screen coordinates and generate the primary ray.
- ▶ `src/scene/scene.cpp`
Traverse primitives and find the closest visible hit.
- ▶ `src/geometry/sphere.cpp`
Implement ray-sphere intersection and write the hit data into `HitRecord`.
- ▶ `src/render/direct_lighting_integrator.cpp`
Compute direct illumination from the point lights.

Focus

Understand where each computation belongs, and connect the pipeline correctly.

Expected Outcome

The default starter scene includes:

- ▶ visible spheres and triangle floor,
- ▶ colors determined by material and direct lighting,
- ▶ hard shadows where objects occlude the light.

What you should learn

- ▶ how camera rays are generated,
- ▶ how intersections are found and stored,
- ▶ how visibility affects lighting,
- ▶ how the renderer passes information from one stage to the next.

